

Translation codes construction supported by computer

Mykola Melnyk

¹ Institute of Multimedia Information and Communication Technologies, FEI STU in Bratislava

xmelnyk@stuba.sk

Abstract – This paper presents the construction and analysis of translation codes using computational methods. We explore both mathematical and algorithmic approaches, including matrix-based encoding and the Freiman-Wyner technique, implemented primarily in Python. Performance comparisons across programming languages highlight trade-offs in speed and memory efficiency. While matrix methods offer theoretical rigor, they are computationally intensive. Experimental improvements to prefix/suffix generation show modest speed gains but remain impractical for real-time use. The results provide a foundation for efficient translation code design in digital communication systems.

1 The importance of the right programming language

Programming language is essential for modern information technology scientific field. It is understandably widely used in different fields as one of the major problem-solving tool. There are an enormous amount of programming languages to choose from, but it certainly depends on task at hand. Some languages are more suitable for certain problems, than others. Therefore, the choice of programming language sometimes decides the flow which algorithms are built, but there are factors that help to make decision easier, such as memory requirements, data amount, time complexity and etc.

In our work we aimed to achieve greater computational speeds for translation codes, while optimizing their usage on machines with any amount of computational power, however. Specifically, to construct our codes using from classic based method, Freiman-Wyner[1] and. Since the amount of data we were going to compute was enormous and there was needed versatility, while still achieving decent computational speeds, the Python was chosen as main language for this task.

1.1 Basics of translation codes construction

Translation codes, itself is group of EC-RLL codes that consist of different encoding and decoding information methods. TC introduce redundancy, in order to translate any digital sequence at the input of TC encoder to such output sequence, which fulfills constraints deduced from practical requirements. To construct TC codes a variety of

methods can be used, therefore each of the methods have its own requirements.

2 Fundamentals of Computerized Construction

When constructing TC codes, there are two methods of computerized construction. In the most common method, mathematical, based on usage of integrated python libraries such as NumPy and SumPy, and calculating matrices while also their roots. Less known, but practically more crude is method based on string concatenation, though the method takes its roots from data science it is still applicable for codeword construction. Both approaches illustrates Fig.1.

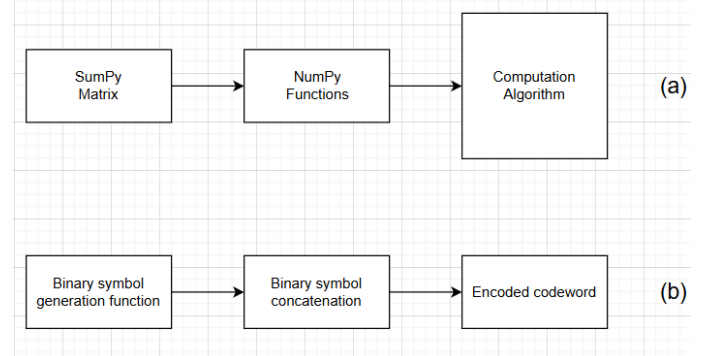


Fig. 1. Computation method: mathematical (a) and crude (b).

Both methods are technically feasible, while possessing certain pros and cons. Mathematical method requires mathematical understanding of algorithms and also library usage, though it significantly improves implementation. The same is true for concatenation method, while majority of scientific data is processed with help of sorting algorithms. Therefore this algorithm can be repurposed for codeword generation. For the matrix construction method there are two existence requirements, that should be met. Such as, the sum of row should never be more or equal to 2^n , where n is equal to the length of encoded codeword. Code speed R should not be greater than Channel capacity C . Where C is equal to Eq.1 and $\lambda_{\max}$ is sum of roots in our matrix.

$$C = \log_2(\lambda_{\max}) \quad (1)$$

The Freiman-Wyner method requires the construction of prefix and suffix, which are constructed by two different means for even or odd scenarios as seen in Eq.2

$$\frac{n}{2} \quad (2)$$

and Eq.3 for odd.

$$\frac{n \pm 1}{2} \quad (3)$$

2.1 Equipment used

Our system uses high-end Personal computer (details see Tab. 1.). Also, JetBrains PyCharm was used as IDE of choice. The Operation System was Windows 11 Pro stable build.

Tab. 1. Technical specifications of the PC

Processor	Intel Core i7-13700k
RAM	Kingston fury ddr5 4800MT/s
Graphics card	Nvidia RTX4080 16GB
Cooling System	NZXT Kraken 280
Storage	Samsung 980 pro 2tb
Operating system	Windows 11 Pro
Motherboard	Gigabyte z790 Aorus
IDE of choice:	
PyCharm	2021.2.3
CLion	2022..3.2

The IDEs were required by applying for student license[2] on JetBrains website.

3 Experimental approach and results

For the purpose of our experiment, we constructed several program iterations of different methods. Therefore, as intended we used Python to recreate matrix method with help of libraries and Freiman-Wyner technique that was realized and tested through concatenation method.

3.1 Freiman-Wyner technique

In this part we tried implementing Freiman-Wyner technique. Therefore, we tried implementing different solutions for computerized construction. The problem at hand we had was that technique itself was rather visual than mathematical. Therefore, the test program was written to test the number of existing and popular programming languages. Program itself is prototype implementation of technique with certain limitations. This script is non-adaptive, meaning the number of prefixes and suffixes cannot be changed during runtime. For the program to function correctly, prefixes and suffixes must be precomputed, and the maximum allowable number of repeated symbols must be defined in advance. The program generates codewords by appending suffixes to a given prefix until two criteria are met: a) The binary symbols in the code do not exceed the specified maximum length constraint. b) The length of the generated codeword matches the desired binary length.

This approach produces more codewords than the original technique, allowing a selection process to identify specific codewords for encoding. Codewords are chosen randomly from the set of valid results.

Then we started our testing phase with implementing technique into C++ programming language. The results are shown on Fig.2.

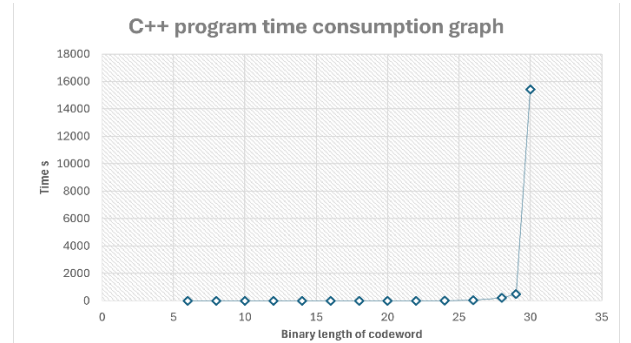


Fig. 2. C++ complete time consumption graph for binary length of 30.

As graph shows us, computation speed significantly decreases at 30 binary digits. Mostly it corresponds to large amounts of operations required and memory limitations.

Although Java's implementation is generally more straightforward due to its user-friendly syntax, the language's inherent memory limitations became apparent during the testing process Fig.3.

```
Exception in thread "main" java.lang.OutOfMemoryError: Create breakpoint : Java heap space
at java.base/java.util.Arrays.copyOf(Arrays.java:3512)
at java.base/java.util.Arrays.copyOf(Arrays.java:3481)
```

Fig. 3. Java memory issue.

The performance analysis, as illustrated in the graph seen in Fig.4., shows that the Python script was most effective among all tested implementations. Specifically, for encoding codewords of 30 binary bits, the script took approximately **2973.1284 seconds**. Which is faster than C++ implementation for the same binary length.

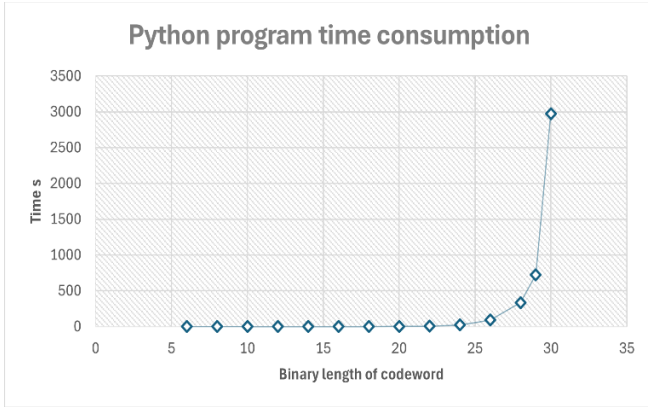


Fig.3. Speed comparison for Python script.

After the initial testing was conducted, we got to the point where this version of Freiman-Wyner technique was not seen effective enough. The peak speed results of each implemented method can be seen in the table below.

Tab. 2. Peak speed comparison table for Implemented solutions.

Programming language	Binary length(n)	Top speed
C++	29	495.48s
Python	30	2973s
Java	20	Not available

Therefore, some experimental solutions were created which is described in next part.

3.2 Experimental approach for Freiman-Wyner

An experimental approach was developed for the Freiman-Wyner encoding technique, utilizing an alternative formula for suffix and prefix generation. Two experiments were conducted, one with division by 4 and the other with division by 8.

The first experiment, which applied division by 4, demonstrated improved computational speeds compared to the classical method that employs a prefix/suffix generation function. Specifically, the computational speed increased by 8.71% relative to the classical approach. However, despite this improvement, the method remains impractical for real-time implementation due to computational constraints (see Fig.4.).

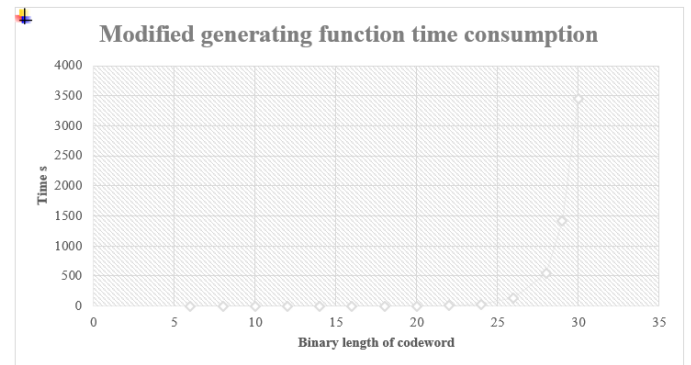


Fig.4. Prefix generation function time consumption graph division by 4.

In contrast, the second experiment, which involved division by 8, resulted in slower computation and a memory overload when generating codewords exceeding 28 binary symbols. The computational speed decreased by 11.65% compared to the first experiment and by 10.96% relative to the classical method (see Fig.5.).

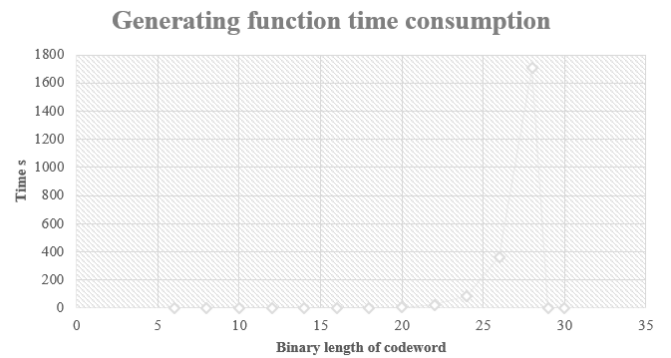


Fig.5. Prefix generation function time consumption graph division by 8.

These results indicate that increasing the division factor beyond 4 is inefficient and impractical for implementation.

3.3 Classic translation code construction (matrix method)

The matrix method for translation code construction was used as a theoretical baseline. It defines allowed symbol transitions through a state matrix and estimates channel capacity via eigenvalue analysis. While mathematically rigorous, this approach proved less practical in real-world scenarios.

Even with a modest number of states, computing higher matrix powers and solving symbolic characteristic polynomials became computationally expensive. Complex roots introduced further numerical challenges, requiring high-precision arithmetic for stable results.

Although the method accurately reflects theoretical limits, it scales poorly and is difficult to implement efficiently. As a result, it is unsuitable for real-time encoding or applications that demand speed and low resource usage.

However, it remains useful as a reference point for evaluating more practical coding techniques.

4 Conclusions

We verified that it is indeed possible to create and adapt translation code construction algorithms for computer-based implementation. With access to high-performance computing systems, it becomes feasible to construct effective translation codes using matrix-based methods and symbolic tools. While the quality and precision of the generated data cannot match that of specialized scientific instruments such as professional spectrometers, the results remain valuable. The proposed method shows strong potential for educational use, prototyping, and future experiments. It also opens up possibilities for further research and advancements in the field of telecommunications, particularly in constrained coding and capacity analysis.

Acknowledgments

We recognize also our supervisor prof. Ing. Peter Farkaš, DrSc. for support with providing all the necessary resources and valuable advice.

References

- [1] Transcontrol codes Peter Farkaš
- [2] <https://www.jetbrains.com/community/education/#students> Educational license JetBrains